

ME100 Final Project - ICBreadboards

Roshni Arun, Jade Stashin, Allison Yuh

December 2025



Contents

1	Project Overview	2
1.1	Problem and Solution	2
1.2	Target Audience	3
1.3	Competitive Analysis	3
2	Prototype	4
2.1	Sensors Overview	4
2.2	Actuators Overview	5
2.3	ESP32s	5
2.3.1	Big Board	5
2.3.2	Mini Board	6
2.3.3	Remote	7
2.3.4	Remote PCB	8
3	Code Architecture	10
4	Manufacturing	11
5	Conclusion	13
5.1	Future Improvements	13
5.2	Acknowledgements	13
6	Appendix	14
6.1	Video of Working Prototype	14
6.2	Remote Code	14
6.3	Big Skateboard Code	18
6.4	Mini Skateboard Code	23
6.5	Overleaf Compiler	30

1 Project Overview

1.1 Problem and Solution

Tracing its origins back to California, skateboards emerged as an alternative mode of micromobility in the early 1960s, with the first wave of commercial skateboards capitalizing on the popularity of surfboards, under the guise of “sidewalking surfing” [1]. Today, skateboarding has distinguished itself from its surf-inspired origins, serving not only as a mode of transportation for nearly 9.3 million Americans in 2024[2], but as a source of recreation and sport, with the rise of pro-skateboarding in the 1970s, and even debuting at the Rio Olympics in 2016. While the first skateboards were disassembled rollerskates mounted on wooden decks, modern innovation has arrived to skateboards with the emergence of electric skateboards.

Advertised as faster and easier to learn than their non-electric predecessors, electric skateboards face a multitude of issues. In general, skateboards are notorious for associated risk. According to the National Safety Council, in 2024, nearly skateboards, scooters, and hoverboards were caused nearly 300,000 injuries alone, 30,000 of which requiring hospitalization and intensive treatment [3]. Moving at significantly higher speeds, reaching as high as 25mph, and capable of abrupt stops, electric skateboards are arguably more dangerous. Additionally, while electric skateboards benefit from their smaller size by being more portable, compared to their electric scooter or bike counterparts, this also puts them at risk of theft.

Importantly, however, electric skateboards take the joy out of skateboarding. While perhaps easier to learn than traditional skateboarding, with the help of a remote control, with no pushing or foot braking, electric skateboards are far less engaging for users to use, and stymies the traditional experience that many skateboards fall in love with in the first place.

Our innovative electric skateboard solution aims to tackle these issues, promising to enhance the riders’ experience by addressing the key frustrations of current boards on the market. Instead of relying on solely remote control, our solution seeks to engage riders through an electrified push mechanism, allowing users to push as they would on normal skateboard, while the built-in motor works to intuitively match and maintain the desired speed. To slow down or stop, riders simply drag their rear foot across the ground, as they would on a traditional skateboard; this will be detected by the board and the on-board motor will slow in response. With the ability to match the users speeds based on traditional skateboarding actions, our solution eliminates the need for a remote entirely, replicating the traditional skateboarding experience while simultaneously harnessing the power of a motorized board. This also enhances user safety, as the board will immediately detect via pressure sensor once the rider’s rear foot is taken off the board and via ultrasonic sensor, is dragged, allowing the board to slow down based on feedback, whereas a typical electric board would not.

In case, however, if the user so desires, our board also can connect to a remote control, like a regular electric skateboard, that through button input, can accelerate and decelerate the board. This remote also comes equipped with a security feature.

With “locked mode” toggled on, when the IMU on the board detects movement (e.g. the board being lifted up), the board’s wheels will lock and alarms built into the board and compassion remote will sound, immediately alerting the user. Thus our skateboard tackles the issues of rider safety, theft, and ride enjoyment.

1.2 Target Audience

Our product targets college students aged 18–24. In 2024, an estimated 14% of this demographic relied on electric skateboards and scooters for campus transportation [4]. We address this market by prioritizing safety through responsive acceleration and braking, while our built-in security system mitigates theft. The result is a reliable commuter vehicle that retains the engagement and fun of a traditional skateboard.

1.3 Competitive Analysis

The demand for micromobility products, the class of small, lightweight personal vehicles, has grown exponentially and is projected to reach USD 91.2 billion by 2030 [4].

While skateboards are well-regarded for their portability and customizability (decks, wheels, etc), they have a steeper learning curve, compared to bikes or scooters, and their speed and ease of use are further limited by terrain (incline, rough sidewalks, etc). Electric skateboards directly solve these limitations by using motorized power, making the riding experience far more accessible for commuters and students.

The market was pioneered by Boosted Board in 2012 with a remote-controlled longboard featuring a remote with LEDs for throttle, battery life, connection, and on/off status. Current leaders like Meepo and Backfire use similar remote-controlled designs. Meepo has built upon Boosted’s original design with a remote featuring an OLED, allowing users to select cruising and braking modes. Backfire added a safety feature of a built-in fingerprint sensor on the remote to prevent the wheels from inadvertently turning if the remote is accidentally pressed.

While most electric skateboards rely on remotes for acceleration and braking, the Liquid Skateboard utilizes push-controlled acceleration to maintain speed. We aim to build on this concept with added security features (e.g. alarms) that are common to electric scooters. Furthermore, our board will pioneer a safe method for riders to receive GPS instructions, a feature no current product offers.

2 Prototype

To validate the feasibility of our non-remote-controlled electric skateboard system while prioritizing user safety, we developed a scaled-down proof-of-concept prototype. Rather than constructing a full motorized skateboard at this stage, our goal was to isolate and test the core interaction paradigm: the ability for the board to interpret traditional skateboarding motions—pushing and foot braking—and translate them into intuitive motor control behavior. Given the inherent risks associated with high-speed motorized systems, we deliberately chose a modular and low-risk prototyping approach. This allowed us to evaluate sensing logic, user interaction, and control responsiveness without exposing riders to unnecessary danger.

2.1 Sensors Overview

The system incorporates sensors distributed across the Big Skateboard and Mini Skateboard for intuitive control, safety, and feedback. An inertial measurement unit (IMU) mounted on the Big Board measures acceleration and motion, allowing the system to detect rider push-off behavior for speed control while also supporting the anti-theft locking mechanism by identifying unauthorized movement. A force-sensitive resistor (FSR) pressure sensor on the Big Board detects foot contact, ensuring that acceleration is only enabled when a rider is properly positioned on the board. An ultrasonic distance sensor provides obstacle and foot-placement detection, enabling foot-cued braking assistance when an object or foot is detected within a defined range. Finally, Hall effect sensors on the Mini Skateboard monitor wheel rotation, supplying speed and acceleration data used for motor control and telemetry. These sensors allow the system to interpret rider intent, monitor environmental conditions, and maintain safe, responsive operation across all modes.

Sensor	Location	Purpose
Hall Sensor	Mini Skateboard	Supplies speed and acceleration data used for motor control and telemetry.
IMU	Big Skateboard	Measures acceleration and motion, allowing the system to detect rider push-off behavior for speed control while also supporting the anti-theft locking mechanism.
FSR Pressure Sensor	Big Skateboard	Foot contact, ensuring that acceleration is only enabled when a rider is properly positioned on the board.
Ultrasonic Sensor	Big Skateboard	Obstacle and foot-placement detection, enabling foot-cued braking assistance.

Table 1: System actuator definitions and locations.

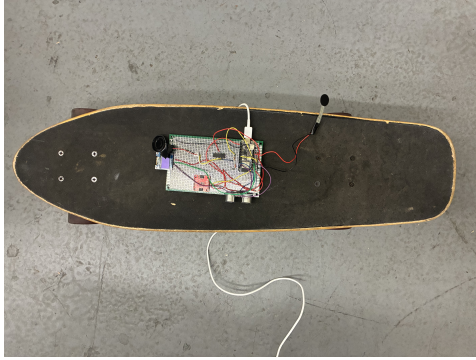


Figure 1: Big Board Prototype

2.2 Actuators Overview

The system comprises five key actuators distributed across three hardware components: the Mini Skateboard, Big Skateboard, and Remote. The 12V DC motor on the Mini Skateboard handles acceleration. User control is centralized on the Remote via three buttons for acceleration, deceleration, and locking. Speed telemetry and locked status are conveyed to the user through a combination of visual indicators (OLED on the Big Board for telemetry; LED on the Remote for lock status) and audio alerts (speakers on both the Remote and Big Board for alarms and acceleration/deceleration button sounds).

Actuator	Location	Purpose
12V DC Motor	Mini Skateboard	Propels board for acceleration; motor speed responds to user actions and remote feedback
OLED Display	Big Skateboard	Speed telemetry and security mode indication
Speakers (2)	Remote (1), Big Skateboard (1)	Theft alarm, audible feedback for button pushing (remote only)
LED	Remote	Security system indication (LED ON = Security Armed, LED OFF = Security Disarmed)
Buttons (3)	Remote	Acceleration, deceleration, and lock/unlock

Table 2: System actuator definitions and locations.

2.3 ESP32s

2.3.1 Big Board

Within the big board, there was an OLED display which displayed the speed, acceleration, whether there was an obstacle detected, and whether the board was armed or disarmed. It has an ESP32 microcontroller and integrated multiple sensors—including an IMU, ultrasonic sensor, and pressure sensor—to continuously

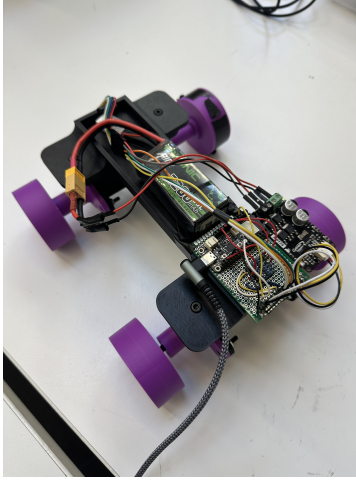


Figure 2: Mini Board Prototype

monitor rider input, board motion, and environmental conditions. The IMU detected acceleration and push-off behavior to inform motor control and also enabled the anti-theft system by triggering alarms when unauthorized motion was detected. The pressure sensor ensured that acceleration was only enabled when a rider's foot was on the board, mimicking natural riding behavior and improving safety. The ultrasonic sensor detected obstacles or foot placement cues to initiate braking assistance. All sensor data was processed on the Big Board and transmitted wirelessly to the Mini Board, where motor commands were executed in real time. The Big Board coordinated sensing, safety, feedback, and communication, while providing clear, real-time feedback to the rider through the OLED display.

2.3.2 Mini Board

The Mini Board serves as the motor controller, receiving wireless commands from the Big Board and Remote to control a 12V DC motor via an H-bridge motor driver. The ESP32 generates a PWM signal on pin 25 (1kHz frequency, 0-1023 duty cycle) to control motor speed, while pin 14 sets the direction. The motor is powered by an external 11.1V LiPo battery that feeds into the motor driver, keeping the motor supply isolated from the ESP32's logic supply. A level shifter interfaces the motor's Hall effect sensors (connected to pins 13 and 12) with the ESP32's 3.3V logic, allowing the controller to read motor position feedback from the 5V Hall sensor outputs. When acceleration commands are received, the motor PWM increases by 2% increments up to maximum duty (100%). When obstacles are detected, the system enters automatic deceleration mode, reducing PWM by 5% every 0.5 seconds until stopped. Safety interlocks prevent acceleration when the system is armed or obstacles are present, ensuring the motor can only run in safe conditions.

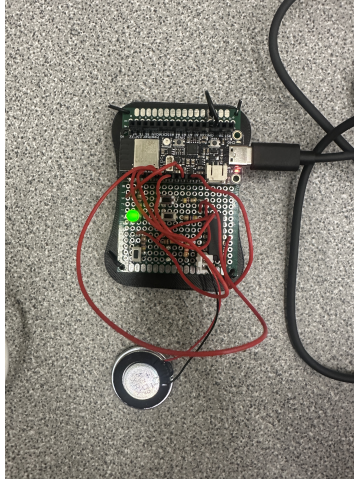


Figure 3: Remote Prototype

Component	Pin Number	Purpose
PWM Output	GPIO 25	Generates PWM signal (1kHz, 0-1023 duty cycle) to control motor speed via H-bridge driver
Direction Control	GPIO 14	Sets motor rotation direction (HIGH/LOW)
Hall Sensor A	GPIO 13	Reads motor position feedback from Hall effect sensor via level shifter (5V to 3.3V)
Hall Sensor B	GPIO 12	Reads motor position feedback from Hall effect sensor via level shifter (5V to 3.3V)
H-Bridge Motor Driver	N/A	Interfaces ESP32 PWM/direction signals with 12V DC motor (powered by 11.1V LiPo battery)
Level Shifter	N/A	Converts Hall sensor 5V outputs to 3.3V logic levels for ESP32 compatibility

Table 3: Mini Board circuit component and pin configuration.

2.3.3 Remote

The remote circuit utilizes an ESP32 Feather with three push buttons (internal pull-ups) for acceleration, deceleration, and toggling security mode. Feedback is provided via a status LED and a PWM-driven speaker. When security mode is engaged, the LED illuminates to confirm the "Locked" state while the ESP32 broadcasts "LOCK" commands. The speaker emits beeps for button presses and functions as a remote alarm; if the ESP32 receives an "ALARM" signal from the main board (indicating theft), it triggers a continuous high-frequency alert.

Component	Pin Number	Purpose
Button (Accelerate)	GPIO 15	Inputs command to increase motor speed (active low)
Button (Decelerate)	GPIO 33	Inputs command to decrease motor speed (active low)
Button (Arm)	GPIO 14	Toggles system security state (Locked/Unlocked)
Speaker	GPIO 27	Generates audio feedback for button presses and theft alarms (PWM)
LED	GPIO 13	Visual indicator for security status (ON = System Armed)

Table 4: Remote circuit component and pin configuration.

2.3.4 Remote PCB

Inspired by our perfboard, we also designed and ordered a custom PCB for the remote (not included in final presentation). The PCB utilizes an ESP32-S2-SOLO-N4CT-ND chip and matches the full capabilities (see BOM) of the perf board remote with the added benefit of a more customized layout and more compact design. The remote is battery-powered and rechargeable by 5V USB-C, employing a buck regulator to step-down the battery power to 3.3V logic. For debugging and physical integration, test points and mounting holes were also added. The schematic and layout below were designed on KiCAD.

Qty (for 1 board)	Part	Manufacturer Part #	Digi-key Part#	Price/unit (\$)	Purchase URL	Reference Designator	Description (include component value if applicable)	Datasheet Link	Multiple	Total Qty
1	esp32 microcontroller chip	ESP32-S2-SOLO	1965-ESP32-S2-1	\$4.25	ESP32-S2-SOLO-U1		ESP Microcontroller; WiFi: 802.11b/g/n Transceiver Module 2.4GHz	https://www.espressif.com/en_US/products/esp32-s2	2	2
1	usbC for battery charging	USB4105-GF-A	2073-USB4105-C	\$0.78	USB4105-GF-A-J1		USB-C (USB TYPE-C) USB 2.0 Receptacle Connector 24 (16+8 Dumm)	https://www.usb.org/developers/connectedsystem/index.php?cid=629&tid=70&cid=629&tid=70	2	2
1	all switch push buttons (10 pack)	CT11028-0F160	2449-CT11028-0	\$1.27	CT11028-0F160	SW1, SW2, SW3, SW4, SW5	Tactile Switch SPST-NO Top Actuated Through Hole	https://www.mouser.com/ProductDetail/CTS/CT11028-0F160	1	1
1	regulator to 3.3V	T1V75739PDBVR	296-S0414-1-NC	\$0.33	T1V75739PDBVR	U3	Linear Voltage Regulator IC Positive Fixed 1 Output 1A SOT-23-5	https://www.ti.com/lit/ds/symlink/t1v75739.pdf	2	2
1	battery charger from 5V	MCP73812T-420IOT	MCP73812T-42	\$0.73	MCP73812T-42	U2	Charger IC Lithium Ion/Polymer SOT-23-5	https://www1.mic.com/products/mcp73812t-42	2	2
2	pin connectors for speaker and battery	S2B-PH-K-5	455-1719-ND	\$0.10	S2B-PH-K-5-35	BAT1,SpeakerC	Connector Header Through Hole, Right Angle 2 position 0.079" (2.0)	https://www.mouser.com/ProductDetail/S2B-PH-K-5-35	2	4
1	220 ohm resistor (10 PACK)	RC0805FR-07220RL	311-220CRCT-NE	\$0.09	RC0805FR-07220RL	R9	220 Ohms ±1% 0.125W, 1/8W Chip Resistor 0805 (2012 Metric) Mol	https://www.onsemi.com/pdf/datasheet/rc0805fr-d.pdf	1	1
1	2.2 k ohm resistor (10 PACK)	RMCF0805FT2K20	RMCF0805FT2K	\$0.06	RMCF0805FT2K20	R4,R7	2.2 kOhms ±1% 0.125W, 1/8W Chip Resistor 0805 (2012 Metric) Aut	https://www.onsemi.com/pdf/datasheet/rmcf0805ft-d.pdf	1	1
1	5.1k ohm resistor (10 PACK)	RMCF0805FT5K10	RMCF0805FT5K	\$0.13	RMCF0805FT5K10	R1,R2	5.1 kOhms ±1% 0.125W, 1/8W Chip Resistor 0805 (2012 Metric) Aut	https://www.onsemi.com/pdf/datasheet/rmcf0805ft-d.pdf	1	1
1	10k ohm resistor (10 PACK)	AF0805FR-0710KL	13-AF0805FR-07	\$0.12	AF0805FR-0710KL	R5, R6, R8, R10,	10 kOhms ±1% 0.125W, 1/8W Chip Resistor 0805 (2012 Metric) Aut	https://www.onsemi.com/pdf/datasheet/af0805fr-d.pdf	1	1
1	0.1 uF capacitor (10 pack)	CL21B104KACNNNC	1276-1099-1-NC	\$0.05	CL21B104KACNNNC	C1,C2,C8	0.1 µF ±10% 25V Ceramic Capacitor X7R 0805 (2012 Metric)	https://www.mouser.com/ProductDetail/CL21B104KACNNNC	1	1
1	1 uF capacitor (10 PACK)	CL181010KAFNNNE	1276-1066-1-NC	\$0.09	CL181010KAFNNNE	C9	1 µF ±10% 25V Ceramic Capacitor X7R 0805 (2012 Metric)	https://www.mouser.com/ProductDetail/CL181010KAFNNNE	1	1
1	4.7 uF capacitor (10 PACK)	GMCC1308473K10NT	4713-04K2135X	\$0.85	GMCC1308473K10NT	C3,C4	4.7 µF ±10% 10V Ceramic Capacitor X5R 0805 (2012 Metric)	https://www.fishchip.com	1	1
1	10 uF capacitor (10 PACK)	CL21A106K0QVNNNE	1276-1096-1-NC	\$0.21	CL21A106K0QVNNNE	C5,C6	10 µF ±10% 16V Ceramic Capacitor X5R 0805 (2012 Metric)	https://www.onsemi.com/pdf/datasheet/cl21a106k0qvnnne.pdf	1	1
1	47 uF capacitor (10 PACK)	GRM32ECR1C476KE15L	490-10531-1-NC	\$3.03	GRM32ECR1C476KE15L	C7	47 µF ±10% 16V Ceramic Capacitor X6S 1210 (3225 Metric)	https://www.onsemi.com/pdf/datasheet/grm32ecr1c476ke15l.pdf	1	1

Figure 4: Remote PCB Components BOM

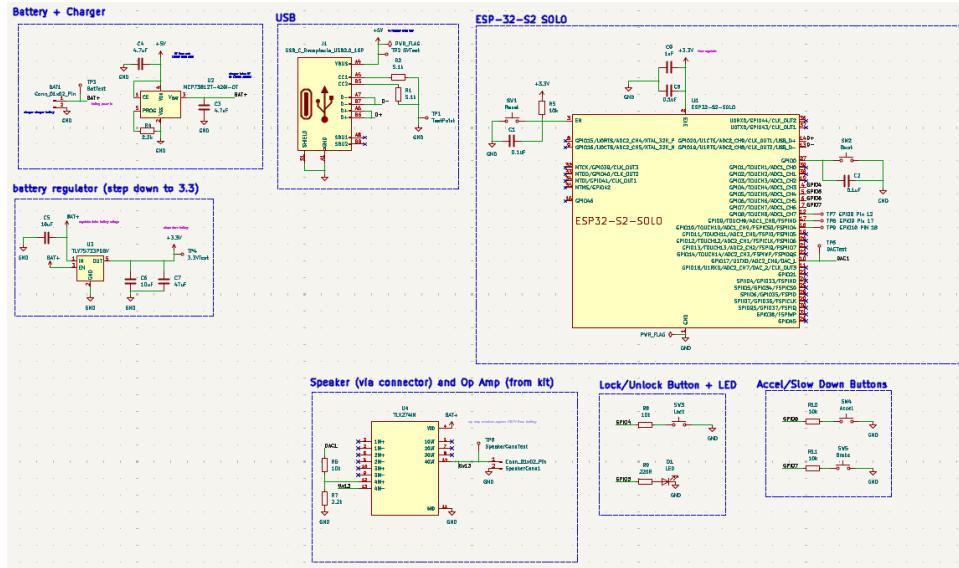


Figure 5: Remote PCB Schematic

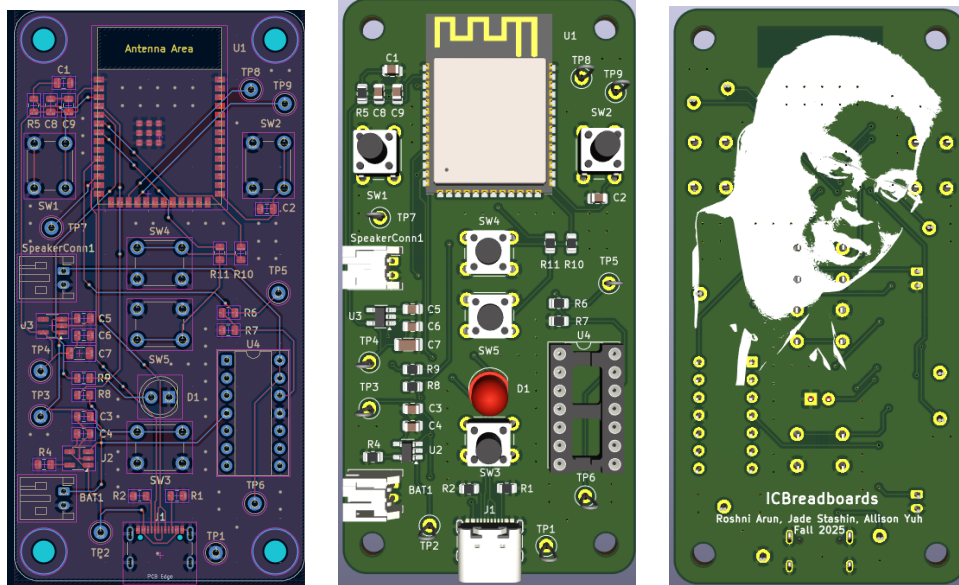


Figure 6: Remote PCB physical design: Board Layout (left), CAD Front view (center), and CAD Back view (right).

3 Code Architecture

With three ESP32s in our working model, we needed a robust and complex communication system to simulate the acceleration, deceleration, and locking of our electric skateboard. Using MicroPython in the Thonny IDE, we used ESP-NOW wireless communication protocol to trigger our actuators. Messages were declared (refer to table), sent, and received by respective ESP32s during key events. Full code is provided in the appendix.

Signal	Sender	Receiver	Purpose
<code>accelerate</code>	Big Board	Mini Board	Increase motor speed by 2% (from pressure sensor)
<code>accelerate_manual</code>	Remote	Mini Board	Increase motor speed by 2% (from button press)
<code>decelerate_slow_start</code>	Big Board	Mini Board	Start automatic 2% per second deceleration
<code>decelerate_slow_stop</code>	Big Board	Mini Board	Stop automatic deceleration
<code>decelerate_manual.decelerate</code>	Remote	Mini Board	Decrease motor speed by 2% once
<code>stop</code>	Big Board	Mini Board	Stop motor immediately
<code>arm</code>	Remote	Mini Board	Disable motor acceleration, stop motor
<code>disarmed</code>	Remote	Mini Board	Re-enable motor acceleration
<code>obstacle_start</code>	Big Board	Mini Board	Disable acceleration (obstacle detected)
<code>obstacle_clear</code>	Big Board	Mini Board	Re-enable acceleration (obstacle cleared)
<code>LOCK</code>	Remote	Big Board	Enter locked/armed mode
<code>UNLOCK</code>	Remote	Big Board	Exit locked mode to ride mode
<code>LED_ON</code>	Big Board	Remote	Turn on LED (system armed)
<code>LED_OFF</code>	Big Board	Remote	Turn off LED (system disarmed)
<code>ALARM</code>	Big Board	Remote	Trigger alarm sound (motion detected while locked)

Table 5: Motor control signal definitions.

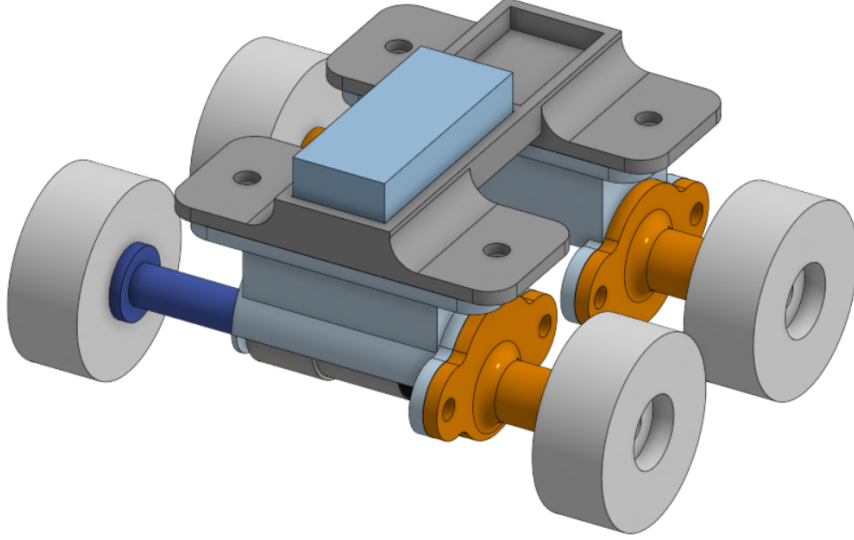


Figure 7: Full mini-skateboard CAD assembly.

4 Manufacturing

The components were assembled using a combination of soldered electronics and 3D-printed mechanical components. All electronic components were soldered onto perf boards, providing a reliable and modular platform for signal routing and power distribution. The mechanical parts, including motor mounts, wheel housings, and structural supports, were designed in OnShape and 3D-printed on a Bambu P1S using PLA filament with a 5% infill, balancing structural strength with material efficiency. Wheels were attached using M6 heat-set inserts and Torx-head shoulder bolts to ensure precise alignment and secure fastening under load. The remaining components were assembled with M4 nuts and bolts for ease of assembly and maintenance. The 12V DC motor's keyed shaft was coupled to a custom friction-fit flange, allowing torque to be transmitted efficiently to the drivetrain while simplifying the mechanical connection. Overall, the combination of 3D printing, modular electronics, and careful hardware selection allowed for rapid prototyping and precise assembly of the system.

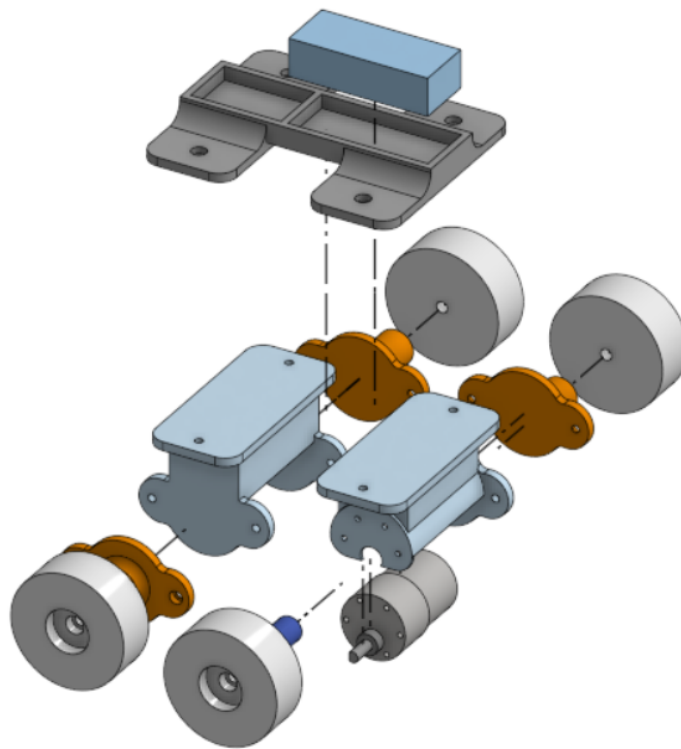


Figure 8: Full mini-skateboard CAD assembly (exploded).

5 Conclusion

5.1 Future Improvements

Future improvements to this project would focus on fully integrating the 36 V BLDC hub motor and battery into a complete, full-scale electric skateboard system. Due to high-voltage safety constraints and challenges getting the larger system operational within the timeline, we validated our approach using a miniature model; however, the next step would be to prototype the full product as a cohesive system before iterating on detailed design refinements. Control strategies could also be significantly improved by moving beyond discrete speed increments to a closed-loop approach that leverages real-time hall sensor data, motor RPM, push-speed estimation, and IMU feedback for smoother, more intuitive acceleration. Additionally, future iterations would incorporate the dedicated remote PCB, which did not arrive in time for this build, and integrate it into a slimmer, more ergonomic remote enclosure. This enclosure would be redesigned with injection molding in mind, enabling improved durability, manufacturability, and overall product polish.

5.2 Acknowledgements

We would like to thank Professor Anwar for all his help and support in this process. We would also like to thank Taiki Yamamoto, Elliot Hong, and Kamron Soltani for their guidance since the beginning of the semester. Lastly, we would love to thank the rest of the Me100 staff for a smooth project development and presentation process.

6 Appendix

6.1 Video of Working Prototype

Video Link

6.2 Remote Code

```
1  import network
2  import espnow
3  from machine import Pin, PWM
4  import time
5
6  # Pins
7  pinAccelerate = 15
8  pinDecelerate = 33
9  pinArm = 14
10 pinSpeaker = 27
11 pinLed = 13
12
13 # Button and LED Inits
14 btnAccelerate = Pin(pinAccelerate, Pin.IN, Pin.PULL_UP)
15 btnDecelerate = Pin(pinDecelerate, Pin.IN, Pin.PULL_UP)
16 btnArm = Pin(pinArm, Pin.IN, Pin.PULL_UP)
17 speaker = PWM(Pin(pinSpeaker), freq=2000, duty=0)
18 led = Pin(pinLed, Pin.OUT)
19 led.off()
20
21 # Vars
22 armed = False
23 alarmSounding = False
24 lastArmPress = 0
25 debounceMs = 300
26
27 sta = network.WLAN(network.STA_IF)
28 sta.active(True)
29
30 e = espnow.ESPNow()
31 e.active(True)
32
33 # Mac Addresses
34 bigBoardMac = b'\xf4e\x0b3\x18\xe0'
35 miniBoardMac = b'\xf4e\x0b4\x1b\x1c'
36
37 e.add_peer(bigBoardMac)
38 e.add_peer(miniBoardMac)
```

```

39
40     print("Remote Control Started")
41     print("MAC:", sta.config('mac').hex(':'))
42
43     def beepSpeaker(durationMs=200, freq=2000):
44         """Beep the speaker"""
45         speaker.freq(freq)
46         speaker.duty(512)
47         time.sleep_ms(durationMs)
48         speaker.duty(0)
49
50     def startAlarmSound():
51         """Start continuous alarm"""
52         speaker.freq(2500)
53         speaker.duty(512)
54
55     def stopAlarmSound():
56         """Stop alarm"""
57         speaker.duty(0)
58
59     def toggleArm():
60         """Toggle armed state"""
61         global armed, alarmSounding
62
63         armed = not armed
64
65         if armed:
66             print("ARMED - LOCK MODE")
67             led.on()
68             e.send(bigBoardMac, b"LOCK")
69             e.send(miniBoardMac, b"arm")
70             beepSpeaker(200, 1200)
71         else:
72             print("DISARMED - RIDE MODE")
73             led.off()
74             e.send(bigBoardMac, b"UNLOCK")
75             e.send(miniBoardMac, b"disarmed")
76             beepSpeaker(200, 700)
77
78             # Stop alarm
79             if alarmSounding:
80                 alarmSounding = False
81                 stopAlarmSound()
82
83     def handleMessage(msg):
84         """Handle incoming messages"""

```

```

85     global alarmSounding
86
87     if msg == b"ALARM" or msg == b"alarm":
88         print("ALARM RECEIVED!")
89         alarmSounding = True
90
91         for _ in range(5):
92             beepSpeaker(50, 1500)
93             time.sleep_ms(50)
94
95         # Then sustained alarm
96         startAlarmSound()
97
98     # Button stuff
99     lastAccelState = 1
100    lastDecelState = 1
101    lastArmState = 1
102
103    try:
104        while True:
105            host, msg = e.recv(0)
106            if msg:
107                print(f"Received: {msg}")
108                handleMessage(msg)
109
110            # button state readout
111            accelState = btnAccelerate.value()
112            decelState = btnDecelerate.value()
113            armState = btnArm.value()
114
115            # Accelerate button
116            if accelState == 0 and lastAccelState == 1:
117                if not armed: # Only allow acceleration when not
118                    armed
119                    print("Accelerate pressed")
120                    e.send(miniBoardMac, b"accelerate")
121                    beepSpeaker(50)
122
123            # Decelerate
124            if decelState == 0 and lastDecelState == 1:
125                if not armed: # Only allow deceleration when not
126                    armed
127                    print("Decelerate pressed")
128                    e.send(miniBoardMac, b"decelerate")
129                    beepSpeaker(50, 900)

```

```

129         # Arm/Disarm
130         if armState == 0 and lastArmState == 1:
131             currentTime = time.ticks_ms()
132             if time.ticks_diff(currentTime, lastArmPress) >
                debounceMs:
133                 toggleArm()
134                 lastArmPress = currentTime
135
136         # Update
137         lastAccelState = accelState
138         lastDecelState = decelState
139         lastArmState = armState
140
141         time.sleep(0.05)
142
143     except KeyboardInterrupt:
144         stopAlarmSound()
145         led.off()
146         e.active(False)
147         sta.active(False)
148         print("Remote Control stopped")

```

6.3 Big Skateboard Code

```
1     import network
2     import espnow
3     from machine import Pin, PWM
4     import time
5
6     # PINS
7     PIN_DIR = 14
8     PIN_PWM = 25
9     PIN_HALL_A = 13
10    PIN_HALL_B = 12
11
12    # MOTOR SETUP
13    motor_dir = Pin(PIN_DIR, Pin.OUT)
14    motor_pwm = PWM(Pin(PIN_PWM), freq=1000, duty=0)
15    motor_dir.value(1) # Set direction
16
17    # Current PWM duty cycle (0-1023)
18    current_duty = 0
19    MAX_DUTY = 1023
20    MIN_DUTY = 0
21
22    # SAFETY
23    alarm_armed = False
24    obstacle_detected = False
25
26    # DECELERATION TRACKING
27    last_decel_time = 0
28    decel_active = False
29    DECEL_RATE = 0.05 # 5% per second
30    DECEL_INTERVAL_MS = 500 # Check every 0.5 second
31
32    # ESP-NOW STUFF
33    sta = network.WLAN(network.STA_IF)
34    sta.active(True)
35
36    e = espnow.ESPNow()
37    e.active(True)
38
39    big_board_mac = b'\xf4\x65\x0b\x33\x18\xe0'
40    remote_mac = b'\xf4\x65\x0b\x31\x08\x24'
41
42    e.add_peer(big_board_mac)
43    e.add_peer(remote_mac)
44
```

```

45 print("Mini Board Started")
46 print("MAC:", sta.config('mac').hex(':'))
47
48 # MOTOR CONTROL FUNCTIONS
49 def set_motor_speed(duty):
50     """Set motor PWM duty cycle (0-1023)"""
51     global current_duty
52     duty = max(MIN_DUTY, min(MAX_DUTY, duty))
53     current_duty = duty
54     motor_pwm.duty(duty)
55     print(f"Motor PWM: {duty} ({duty/MAX_DUTY*100:.1f}%)")
56
57 def accelerate():
58     """Increase motor speed by 2% (from big board)"""
59     global current_duty, decel_active
60     if alarm_armed:
61         print("Cannot accelerate: Alarm is armed")
62         return
63     if obstacle_detected:
64         print("Cannot accelerate: Obstacle detected")
65         return
66
67     decel_active = False
68
69     increase = int(MAX_DUTY * 0.02)
70     new_duty = current_duty + increase
71     set_motor_speed(new_duty)
72
73 def accelerate_manual():
74     """Increase motor speed by 2% (from remote)"""
75     global current_duty, decel_active
76     if alarm_armed:
77         print("Cannot accelerate: Alarm is armed")
78         return
79     if obstacle_detected:
80         print("Cannot accelerate: Obstacle detected")
81         return
82
83     decel_active = False
84
85     increase = int(MAX_DUTY * 0.02)
86     new_duty = current_duty + increase
87     set_motor_speed(new_duty)
88
89 def decelerate_slow_start():

```

```

90         """Start automatic deceleration at 2% per second (from big
91             board)"""
92     global decel_active, last_decel_time
93     decel_active = True
94     last_decel_time = time.time()
95     print("Automatic deceleration started (2% per second)")
96
97     def decelerate_slow_stop():
98         """Stop automatic deceleration"""
99         global decel_active
100        decel_active = False
101        print("Automatic deceleration stopped")
102
103    def decelerate_manual():
104        """Decrease motor speed by 2% once (from remote)"""
105        global current_duty, decel_active
106
107        decel_active = False
108
109        decrease = int(MAX_DUTY * 0.02)
110        new_duty = current_duty - decrease
111        set_motor_speed(new_duty)
112
113    def stop_motor():
114        """Stop the motor immediately"""
115        global decel_active
116        decel_active = False
117        set_motor_speed(0)
118        print("Motor stopped")
119
120    def handle_auto_deceleration():
121        """Handle automatic deceleration if active"""
122        global last_decel_time, decel_active, current_duty
123
124        if decel_active and current_duty > 0:
125            current_time = time.time()
126            if time.time_diff(current_time, last_decel_time) >=
127                DECEL_INTERVAL_MS:
128                if current_duty <= 20:
129                    set_motor_speed(0)
130                    decel_active = False
131                    print("Automatic deceleration complete - motor
132                        stopped")
133                else:
134                    decrease = int(MAX_DUTY * DECEL_RATE)
135                    new_duty = current_duty - decrease

```

```

133
134         if new_duty <= 0:
135             set_motor_speed(0)
136             decel_active = False
137             print("Automatic deceleration complete - motor
138                 stopped")
139         else:
140             set_motor_speed(new_duty)
141
142     last_decel_time = current_time
143
144 def handle_message(msg):
145     global alarm_armed, obstacle_detected
146
147     if msg == b"accelerate":
148         accelerate()
149
150     elif msg == b"accelerate_manual":
151         accelerate_manual()
152
153     elif msg == b"decelerate_slow" or msg ==
154         b"decelerate_slow_start":
155         decelerate_slow_start()
156
157     elif msg == b"decelerate_slow_stop":
158         decelerate_slow_stop()
159
160     elif msg == b"decelerate_manual" or msg == b"decelerate":
161         decelerate_manual()
162
163     elif msg == b"stop":
164         stop_motor()
165
166     elif msg == b"arm":
167         alarm_armed = True
168         stop_motor() # Stop motor when arming
169         print("Alarm armed - motor stopped, acceleration disabled")
170
171     elif msg == b"disarmed":
172         alarm_armed = False
173         print("Alarm disarmed - acceleration enabled")
174
175     elif msg == b"obstacle_start":
176         obstacle_detected = True
177         print("Obstacle detected - acceleration disabled")

```

```

177     elif msg == b"obstacle_clear":
178         obstacle_detected = False
179         print("Obstacle cleared - acceleration enabled")
180
181     # MAIN LOOP
182     try:
183         while True:
184             host, msg = e.recv(0)
185             if msg:
186                 print(f"Received: {msg}")
187                 handle_message(msg)
188
189             handle_auto_deceleration()
190
191             time.sleep(0.05)
192
193     except KeyboardInterrupt:
194         stop_motor()
195         e.active(False)
196         sta.active(False)
197         print("Mini Board stopped")
198
199

```

6.4 Mini Skateboard Code

```
1     import network
2     import espnow
3     from machine import Pin, PWM, I2C, ADC
4     import time
5     from time import ticks_ms
6     from lsm6dsox import LSM6DSOX
7     import ssd1306
8     from hcsr04 import HCSR04
9
10
11     # PIN CONFIGURATION
12
13     PIN_SPEAKER = 27
14     PIN_IMU_SDA = 22
15     PIN_IMU_SCL = 14
16     PIN_ULTRASONIC_TRIG = 7
17     PIN_ULTRASONIC_ECHO = 8
18     PIN_PRESSURE = 36
19     OLED_SCL = 32
20     OLED_SDA = 15
21     OLED_ADDR = 0x3C
22
23
24     # WHEEL / SPEED CONSTANTSkjjkk
25
26     WHEEL_DIAMETER_M = 0.065
27     WHEEL_CIRC_M = 3.1416 * WHEEL_DIAMETER_M
28     MAX_RPM = 300
29     PWM_MAX = 1023
30     current_pwm = 0
31
32
33     # MODES
34
35     MODE_RIDE = 0
36     MODE_LOCKED = 1
37     current_mode = MODE_RIDE
38
39
40     # THRESHOLDS
41
42     ULTRASONIC_THRESHOLD_CM = 25.0
43     ACCEL_THRESHOLD = 0.05
44     LOCK_GYRO_THRESHOLD = 100.0
```

```

45 LOCK_ACCEL_THRESHOLD = 0.3
46 PRESSURE_THRESHOLD = 1
47 ALARM_COOLDOWN_TIME = 2.0
48
49
50 # HARDWARE SETUP
51
52 speaker = PWM(Pin(PIN_SPEAKER), freq=2000, duty=0)
53
54 ultrasonic = HCSR04(
55     trigger_pin=PIN_ULTRASONIC_TRIG,
56     echo_pin=PIN_ULTRASONIC_ECHO,
57     echo_timeout_us=30000
58 )
59
60 pressure_adc = ADC(Pin(PIN_PRESSURE))
61 pressure_adc.atten(ADC.ATTN_11DB)
62 pressure_adc.width(ADC.WIDTH_12BIT)
63
64 # OLED
65 try:
66     i2c_oled = I2C(1, scl=Pin(OLED_SCL), sda=Pin(OLED_SDA),
67         freq=400000)
68     oled = ssd1306.SSD1306_I2C(128, 32, i2c_oled, OLED_ADDR)
69 except:
70     oled = None
71
72 # IMU
73 i2c_imu = I2C(0, scl=Pin(PIN_IMU_SCL), sda=Pin(PIN_IMU_SDA),
74     freq=400000)
75 try:
76     imu = LSM6DSOX(i2c_imu)
77 except:
78     imu = None
79
80 # ESP-NOW
81
82 sta = network.WLAN(network.STA_IF)
83 sta.active(True)
84
85 e = espnow.ESPNow()
86 e.active(True)
87
88 mini_board_mac = b'\xf4e\x0b3"\xd4'
89 lock_board_mac = b'\xf4e\x0b1\x08$'

```

```

89
90 e.add_peer(mini_board_mac)
91 e.add_peer(lock_board_mac)
92
93 print("Big Board Started")
94 print("MAC:", sta.config('mac').hex(':'))
95
96
97 # IMU CALIBRATION
98
99 print("Calibrating IMU... keep sensor still")
100 calibration_samples = 20
101 gyro_offset = [0, 0, 0]
102 accel_offset = [0, 0, 0]
103
104 if imu:
105     for i in range(calibration_samples):
106         try:
107             ax, ay, az = imu.accel()
108             gx, gy, gz = imu.gyro()
109             gyro_offset[0] += gx
110             gyro_offset[1] += gy
111             gyro_offset[2] += gz
112             accel_offset[0] += ax
113             accel_offset[1] += ay
114             accel_offset[2] += az
115             time.sleep(0.05)
116         except:
117             pass
118
119     gyro_offset = [x / calibration_samples for x in gyro_offset]
120     accel_offset = [x / calibration_samples for x in accel_offset]
121     print(f"Calibration complete. Gyro offset: {gyro_offset}")
122     print(f"Accel offset: {accel_offset}")
123
124
125 # UTILITY FUNCTIONS
126
127 def pwm_to_speed_mps(pwm):
128     rpm = (pwm / PWM_MAX) * MAX_RPM
129     speed = (rpm * WHEEL_CIRC_M) / 60
130     return rpm, speed
131
132 def read_imu_accel():
133     try:
134         return imu.accel()

```

```

135         except:
136             return 0.0, 0.0, 9.8
137
138     def read_imu_gyro():
139         try:
140             return imu.gyro()
141         except:
142             return 0.0, 0.0, 0.0
143
144     def read_pressure():
145         return pressure_adc.read()
146
147     def read_ultrasonic_raw():
148         try:
149             cm = ultrasonic.distance_cm()
150             mm = ultrasonic.distance_mm()
151             if cm <= 0 or cm > 400:
152                 return None, None
153             return cm, mm
154         except:
155             return None, None
156
157
158     # ALARM CONTROL
159
160     def start_alarm():
161         speaker.freq(2500)
162         speaker.duty(512)
163
164     def stop_alarm():
165         speaker.duty(0)
166
167     def beep(freq=1000, duration=0.1):
168         """Play a short beep on the speaker."""
169         speaker.freq(freq)
170         speaker.duty(512)
171         time.sleep(duration)
172         speaker.duty(0)
173
174
175     # OLED UPDATE
176
177     def oled_update(speed_pwm, accel_mag, object_detected, locked=False,
178                     alarm=False):
179         if oled is None:
180             return

```

```

180
181     oled.fill(0)
182
183     if locked:
184         oled.text("STATUS: LOCKED", 0, 0)
185         oled.text("!! MOTION !!" if alarm else "Armed - Stable", 0,
186                   16)
187         oled.show()
188         return
189
190     _, speed = pwm_to_speed_mps(speed_pwm)
191     oled.text(f"Speed:{speed:.2f}m/s", 0, 0)
192     oled.text(f"Accel:{accel_mag:.2f}g", 0, 10)
193     oled.text("OBJECT DETECTED" if object_detected else "Path
194               Clear", 0, 22)
195     oled.show()
196
197
198     # PRESSURE CALIBRATION
199
200     def calibrate_pressure():
201         samples = []
202         for _ in range(50):
203             samples.append(read_pressure())
204             time.sleep(0.02)
205         return (sum(samples) / len(samples)) + 100
206
207
208     # ESP-NOW HANDLER
209
210     alarm_active = False
211     last_alarm_time = 0
212
213     def handle_espnw():
214         global current_mode, current_pwm, alarm_active, last_alarm_time
215
216         if e.any():
217             host, msg = e.recv()
218
219             if msg == b'LOCK' or msg == b'arm':
220                 current_mode = MODE_LOCKED
221                 current_pwm = 0
222                 alarm_active = False
223                 stop_alarm()
224                 e.send(mini_board_mac, b"stop")
225                 e.send(lock_board_mac, b"LED_ON")

```

```

224         last_alarm_time = time.time()
225         print("LOCKED MODE ACTIVATED")
226
227     elif msg == b'UNLOCK' or msg == b'disarmed':
228         current_mode = MODE_RIDE
229         alarm_active = False
230         stop_alarm()
231         e.send(lock_board_mac, b"LED_OFF")
232         print("RIDE MODE ACTIVATED")
233
234
235     # RIDE MODE
236
237     previous_object_detected = False
238
239     def ride_mode_loop():
240         global current_pwm, previous_object_detected
241
242         ax, ay, az = read_imu_accel()
243         accel_mag = (ax**2 + ay**2 + az**2) ** 0.5 - 1.0
244         pressure = read_pressure()
245         distance_cm, _ = read_ultrasonic_raw()
246
247         print(
248             f"{ticks_ms()} | accel={accel_mag:.2f}g |
249             pressure={pressure} | dist={distance_cm}"
250         )
251
252         object_detected = (
253             distance_cm is not None and
254             distance_cm < ULTRASONIC_THRESHOLD_CM
255         )
256
257         # Handle obstacle state changes
258         if object_detected and not previous_object_detected:
259
260             print("OBSTACLE DETECTED - Starting deceleration")
261             e.send(mini_board_mac, b"obstacle_start")
262             e.send(mini_board_mac, b"decelerate_slow_start")
263         elif not object_detected and previous_object_detected:
264
265             print("OBSTACLE CLEARED - Stopping deceleration")
266             e.send(mini_board_mac, b"obstacle_clear")
267             e.send(mini_board_mac, b"decelerate_slow_stop")
268

```

```

269     previous_object_detected = object_detected
270
271     oled_update(current_pwm, accel_mag, object_detected)
272
273     # Normal acceleration from pressure sensor
274     if not object_detected and abs(ay) > ACCEL_THRESHOLD and
        pressure > PRESSURE_THRESHOLD:
275         current_pwm = int(0.7 * PWM_MAX)
276         e.send(mini_board_mac, b"accelerate")
277
278
279     # LOCKED MODE (ARMED)
280
281     def locked_mode_loop():
282         global alarm_active, last_alarm_time
283
284         ax, ay, az = read_imu_accel()
285         gx, gy, gz = read_imu_gyro()
286
287
288         gx -= gyro_offset[0]
289         gy -= gyro_offset[1]
290         gz -= gyro_offset[2]
291
292
293         gyro_magnitude = (gx**2 + gy**2 + gz**2)**0.5
294
295
296         ax_dev = abs(ax - accel_offset[0])
297         ay_dev = abs(ay - accel_offset[1])
298         az_dev = abs(az - accel_offset[2])
299         accel_magnitude = (ax_dev**2 + ay_dev**2 + az_dev**2)**0.5
300
301         print(f"[LOCKED] Gyro: {gyro_magnitude:.2f}/s | Accel dev:
        {accel_magnitude:.3f}g")
302
303         current_time = time.time()
304         time_since_last_alarm = current_time - last_alarm_time
305
306
307         motion_detected = (gyro_magnitude > LOCK_GYRO_THRESHOLD or
        accel_magnitude > LOCK_ACCEL_THRESHOLD)
308
309
310         if motion_detected and not alarm_active and
        time_since_last_alarm > ALARM_COOLDOWN_TIME:
311             print("MOTION DETECTED - TRIGGERING ALARM!")

```

```

312         alarm_active = True
313         start_alarm()
314         e.send(lock_board_mac, b"ALARM")
315         last_alarm_time = current_time
316
317
318         for _ in range(5):
319             beep(1500, 0.05)
320             time.sleep(0.05)
321
322         start_alarm()
323
324     oled_update(0, gyro_magnitude, False, locked=True,
325               alarm=alarm_active)
326     time.sleep(0.1)
327
328
329
330 try:
331     PRESSURE_THRESHOLD = calibrate_pressure()
332     print(f"Pressure threshold calibrated: {PRESSURE_THRESHOLD}")
333
334     while True:
335         handle_espnw()
336
337         if current_mode == MODE_RIDE:
338             ride_mode_loop()
339         else:
340             locked_mode_loop()
341
342         time.sleep(0.1)
343
344 except KeyboardInterrupt:
345     stop_alarm()
346     e.active(False)
347     sta.active(False)
348     if oled:
349         oled.fill(0)
350         oled.show()
351     print("Big Board stopped")
352

```

6.5 Overleaf Compiler

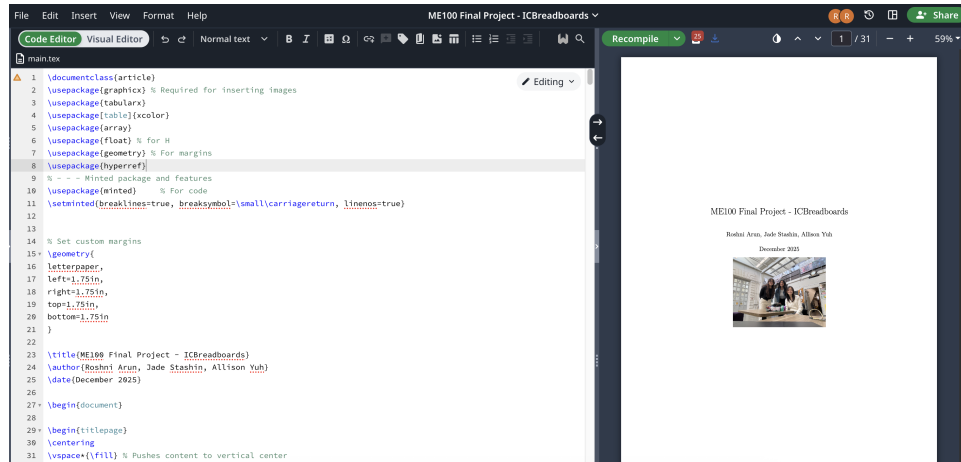


Figure 9: Overleaf Compiler

References

- [1] <https://www.britannica.com/sports/skateboarding>
- [2] <https://www.statista.com/statistics/191308/participants-in-skateboarding-in-the-us-since-2006/>
- [3] <https://injuryfacts.nsc.org/home-and-community/safety-topics/sports-and-recreational-injuries>
- [4] <https://www.grandviewresearch.com/industry-analysis/micro-mobility-market-report>